

SPECCEM FuseEval-lite 复现中的关键模块说明

Reproduction notes

2026 年 5 月

说明

本文档回答四个问题：(1) FuseEval 是什么；(2) 本次 SPECCEM 管线如何设置，以及它与原论文设置的全部关键区别；(3) 本次测评过程的数据管线；(4) 本次复现涉及的指标含义、计算方式和数学公式。文档依据本地论文摘录、复现仓库配置和代码实现整理，重点解释本次复现中真实采用的流程，而不是只复述原论文。

目录

1 FuseEval 是什么？	2
1.1 FuseEval 在原论文中的定位	2
1.2 本次复现中的 FuseEval-lite	2
1.3 为什么不是完整 FuseEval	3
2 本次 SPECCEM 管线如何设置？和原论文全部区别在哪里？	3
2.1 原论文 SPECCEM 的关键设置	3
2.2 本次复现的核心设置	4
2.3 模型加载与生成适配层：问题、原因和解决方案	4
2.4 SpecEM-4-prefused 的融合逻辑	6
2.5 本次设置与原论文设置的系统性区别	8
3 测评过程的详细数据管线是什么？	9
3.1 总体数据流	9
3.2 数据样本格式	9
3.3 验证子集与质量门控	10
3.4 单模型完整生成	11
3.5 SpecEM-4-prefused 融合	11
3.6 最终结果文件	11

4 几个指标分别代表什么？怎么算？数学公式是什么？	12
4.1 本次实际采用的指标	12
4.2 BLEU / sacreBLEU	12
4.3 ROUGE-N: ROUGE-1 与 ROUGE-2	13
4.4 ROUGE-L	13
4.5 ROUGE-Lsum	14
4.6 质量门控指标	14
4.7 原论文中涉及但本次未最终采用的指标	15
5 总结	15

1 FuseEval 是什么？

1.1 FuseEval 在原论文中的定位

FuseEval 是 SPECEM 论文用于评估开放生成能力的多语言指令响应 benchmark。它的目标不是考察选择题正确率，也不是数学推理 exact match，而是考察模型在给定自然语言指令后生成自由文本回答的质量。因此，FuseEval 的输出不是 A/B/C/D 选项，而是一段自然语言响应；评估时通常把模型生成回答与参考回答进行比较，或者用强模型/人工偏好评测进行排序。

根据本地论文摘录，原论文构造 FuseEval 的方式是：

- 英文部分：组合 Dolly-15k 和 Alpaca-GPT4。
- 中文部分：来自 COIG-CQIA 中的 Human-Value 和 Ruozb。
- 评价指标：BARTScore、BERTScore、BLEU、ROUGE、GPT4-Rank。
- 中文 BLEU 和 ROUGE：原论文说明在计算前使用 Jieba 分词。

因此，FuseEval 可以理解为一个双语开放生成测试集：每条样本包含一条指令或问题、一个参考回答，以及模型需要生成的回答。

1.2 本次复现中的 FuseEval-lite

本次复现没有完整复刻原论文 FuseEval 全量数据，而是构建了一个可在单卡条件下运行的 FuseEval-lite 子集。该子集共 400 条样本：

- 英文 200 条；
- 中文 200 条；
- 处理后文件为 `specem_repro/data/processed/fuseeval_lite.json`；

- 最终所有模型输出均按这 400 条样本对齐。

本次 FuseEval-lite 的定位是：在可承受成本和单张 RTX 4090 环境下，复现 SPECEM 论文中 FuseEval 开放生成部分的核心现象，即单模型基线与 SPECEM 风格集成输出在参考指标上的表现差异。

1.3 为什么不是完整 FuseEval

完整复现原论文 FuseEval 设置需要更高硬件资源、更完整数据构造、更严格的原始模型矩阵和多次随机运行。原论文在 7B-9B 实验中使用 A100，在更大模型实验中使用 H200，并对结果做三次独立运行平均。本次复现的目标是单卡、可发布、可复算，因此采用 FuseEval-lite 作为核心复现范围。

2 本次 SPECEM 管线如何设置？和原论文全部区别在哪里？

2.1 原论文 SPECEM 的关键设置

根据论文摘录，原论文 SPECEM 具有以下关键设置：

- 方法：训练无关、推理时集成，包含 drafting、verification 和 online feedback。
- 模型并行：多个模型在推理过程中协作，进行 segment-level 生成和验证。
- 7B-9B 基础模型：Llama-3-8B-Instruct、Mistral-7B-Instruct-v0.3、Qwen2-7B-Instruct、GLM-4-9B-Instruct、Gemma-2-9B-Instruct。
- 大模型扩展：Qwen2-72B、Llama3-70B、Qwen2.5-32B、Mistral-24B-Instruct-2501 等。
- 精度与硬件：bfloat16；7B-9B 使用 A100，大模型使用 H200。
- 生成配置：do_sample=True, temperature=0.6, top_p=0.9。
- segment 设置：候选 segment 数最大值 $L = 10$ 。
- online feedback 超参数： $\alpha = 1$ 。
- 结果统计：三次独立运行取平均。
- FuseEval 指标：ROUGE、BLEU、BARTScore、BERTScore、GPT4-Rank；中文 BLEU/ROUGE 使用 Jieba 分词。

2.2 本次复现的核心设置

本次复现采用的是单卡可运行的 **SpecEM-4-prefused** 近似管线，而不是原论文完整在线 SPECEM。核心设置如下：

- 硬件：单张 NVIDIA RTX 4090，24GB 显存。
- 加载方式：模型顺序加载，不同时驻留；使用 4-bit NF4 量化以适配显存。
- 数据：FuseEval-lite 400 条双语样本。
- 基础模型：Gemma-3-12B-it、Xiaomi MiMo-7B-SFT、Qwen3.5-9B、Ministral-3-8B-Instruct-2512。
- 单模型生成：先逐个模型生成完整回答，保存为 JSONL。
- 集成方式：读取四个单模型的已生成回答，对每个样本做离线片段融合。
- segment 长度：默认 `segment_tokens=48`。
- online 更新系数：默认 `eta=0.35`。
- 指标：sacreBLEU、ROUGE-1、ROUGE-2、ROUGE-L、ROUGE-Lsum。
- 质量门控：行数、语言覆盖、空输出、reasoning 泄漏、token cap 饱和。

2.3 模型加载与生成适配层：问题、原因和解决方案

本次复现中，模型加载与生成适配层不是附属工程细节，而是决定实验是否可信的关键部分。表面上，这些模型都“生成文字 token”；但它们的 loader class、chat template、EOS 语义、reasoning 通道和显存占用方式并不相同。如果直接用统一的 `AutoModelForCausalLM` + 默认 prompt 方式运行，得到的结果会混入接口错误、隐藏推理泄漏或 token cap 饱和，不能代表模型真实能力。

统一适配层的总体设计。 本次实现将模型加载和生成封装为统一的 `UnifiedTextGenerator`。它先根据模型名称识别 family，例如 Qwen、Gemma、MiMo、Mistral；随后优先尝试 `AutoModelForCausalLM`，如果失败，再回退到 `AutoProcessor` + `AutoModelForImageTextToText`。所有模型生成前都会通过统一入口构造 inputs，生成后只截取 prompt 之后的新 token：

$$\hat{y} = \text{Decode}(\text{output}[\ell_{\text{input}} :]),$$

其中 ℓ_{input} 是输入 prompt 的 token 长度。这样可以避免把 prompt 本身误算进模型回答。

问题一：Qwen3.5 thinking 泄漏。 现象是 Qwen3.5 在早期运行中会输出 hidden-reasoning 风格文本，或者生成大量分析过程，导致回答不再是 FuseEval 需要的“最终答案”。这种输出会直接污染 BLEU/ROUGE：一方面它会拉长输出，降低 BLEU precision；另一方面它可能因为包含部分题面或解释性词汇而虚高某些 overlap 指标。

原因在于新一代 Qwen 模型可能支持 thinking/reasoning 模式。如果 chat template 没有显式关闭 thinking，模型会把推理过程作为普通可见文本输出。解决方案是在调用 `apply_chat_template` 时检测是否支持 `enable_thinking` 参数；如果支持，就设置：

```
enable_thinking=False
```

若 signature 检测不到该参数，但模型 family 是 Qwen，则仍尝试传入 `enable_thinking=False`。同时，system prompt 明确要求只输出最终答案，不输出 reasoning steps、thinking process 或 hidden thoughts。质量门控用 `thinking_rate` 检测 `<think>`、thinking process、analyze the request 等标记。最终 Qwen3.5 的 thinking leakage 被降为 0。

问题二：Gemma EOS list。 现象是部分模型已经给出完整答案后仍继续生成，直到达到最大 token 限制。若这种情况发生，token-cap 率会上升，输出中可能出现冗余文本，进而影响 ROUGE 和 BLEU 的解释。

原因是 Gemma-3-12B-it 等模型的 EOS 配置可能不是单个 token id，而是一个 EOS list。如果代码把 EOS list 错误地压成单个标量，模型就可能无法在某些合法结束符处停止。解决方案是在生成前从三个位置依次读取 EOS 候选：

```
generation_config.eos_token_id,model.config.eos_token_id,tokenizer.eos_token_id
```

如果 EOS 是 list 或 tuple，则保留其中所有非空 token id：

$$E = \{e_1, e_2, \dots, e_k\}.$$

生成时把整个 E 传给 `model.generate`，而不是只传一个 e_i 。pad token 若缺失，则使用 EOS 作为 fallback。这样可以让模型在任一合法 EOS 上停止，降低虚假 token cap。

问题三：MiMo reasoning channel。 现象是 MiMo-7B-SFT 初始输出容易包含 `<think>` 或推理过程。即使最终答案是正确的，公开推理文本也会破坏开放生成评测，因为参考答案通常只包含最终回答，不包含模型内部思考。

原因是 MiMo 是 reasoning-tuned 模型，并且模型卡建议使用空 system prompt；如果普通 system prompt 与 reasoning 模板冲突，模型更容易把 reasoning channel 输出到 answer channel。解决方案分两层。第一，MiMo 的 message 结构使用空 system prompt，把“直接回答、不要输出 `<think>`、不要写 reasoning/analysis/planning”等约束放入 user message。第二，在 causal LM 模式下使用专门的 plain MiMo prompt，显式预关闭 reasoning 区域：

```
<think>\n\n</think>\n\n
```

这样模型进入 assistant answer 区域时, reasoning channel 已经被关闭。生成后还会用 `clean_generated_text` 删除残留的 `<think>...</think>` 片段, 并在质量门控阶段检查 reasoning 泄漏。

问题四: ImageTextToText loader。 现象是部分新模型虽然最终输出文本, 但不能稳定用 `AutoModelForCausalLM` 加载, 或者需要 `processor` 而不是 `tokenizer` 来构造输入。若强行统一使用 `CausalLM` 路径, 可能直接加载失败, 或生成输入格式错误。

原因是 Transformers 中“生成文字”不等于“一定是 `CausalLM` 类”。某些 instruction 模型、视觉语言模型或新架构模型暴露为 `image-text-to-text` 接口, 即使本实验只传文本, 也需要通过 `AutoProcessor` 和 `AutoModelForImageTextToText` 构造合法输入。解决方案是 loader fallback:

`AutoModelForCausalLM` $\rightarrow$ `AutoProcessor` + `AutoModelForImageTextToText`

在 `image-text-to-text` 模式下, 代码使用 `processor` 的 `apply_chat_template` 构造文本, 再调用 `processor(text=..., return_tensors="pt")` 得到 `input_ids` 和相关 `tensor`。统一的 `_normalize_inputs` 会保留所有带 `shape` 且可迁移到 `device` 的 `tensor`, 包括 `attention mask`, 从而避免输入字段缺失。

问题五: 4-bit 量化与顺序加载。 现象是原论文 7B-9B 实验依赖 A100, 大模型实验依赖 H200; 而本地只有单张 24GB RTX 4090, 不可能同时加载四个 7B-14B 级模型, 更不可能以 `bf16` 并行执行原论文在线 SPECEM。

解决方案是把并行在线推理改造成顺序单模型生成 + 离线 `prefused` 融合。每次只加载一个模型, 使用 `bitsandbytes 4-bit NF4` 量化:

`load_in_4bit=True, bnb_4bit_quant_type="nf4", bnb_4bit_compute_dtype=bfloat16`

模型运行完成后调用卸载和缓存清理, 释放显存与磁盘。MiMo 在验证脚本中还额外将 `max_new_tokens` 限制为 128, 以降低 `reasoning-tuned` 模型长输出带来的 `token cap` 风险。该策略牺牲了与原论文在线多模型交互的一致性, 但换来了单卡可运行、可复算、可发布的实验管线。

为什么这些适配会影响结论。 这些问题如果不处理, 会把“接口失败”误当成“模型能力差”。例如, Qwen3.5 修复前看起来异常弱, 修复后成为 `sacreBLEU` 最高的系统; Gemma EOS 处理错误会制造虚假的 `token cap`; MiMo reasoning 泄漏会让输出包含参考答案不需要的推理过程; `ImageTextToText loader` 错误会导致模型无法正常进入生成; 不做 4-bit 顺序加载则实验根本无法在单张 RTX 4090 上完成。因此, 本次复现中的模型适配层实质上是实验方法的一部分, 而不是普通工程包装。

2.4 SpecEM-4-prefused 的融合逻辑

设某个样本上四个模型输出为

$$Y = \{y_1, y_2, y_3, y_4\}.$$

每个输出 y_m 被切分为若干片段：

$$y_m = (s_{m,1}, s_{m,2}, \dots, s_{m,T_m}).$$

在第 t 个片段位置，活跃候选集合为

$$A_t = \{s_{m,t} : t \leq T_m\}.$$

对于某个候选片段 $s_{m,t}$ ，本次实现首先计算它与其他活跃片段的词面重叠分数。代码中把文本 token 化为中文单字或英文/数字片段，然后计算 candidate 与 peer 的 multiset overlap。对某个 peer p ：

$$\begin{aligned} \text{Prec}(s, p) &= \frac{|C(s) \cap C(p)|}{|C(s)|}, \\ \text{Rec}(s, p) &= \frac{|C(s) \cap C(p)|}{|C(p)|}, \\ F_1(s, p) &= \frac{2 \cdot \text{Prec}(s, p) \cdot \text{Rec}(s, p)}{\text{Prec}(s, p) + \text{Rec}(s, p)}. \end{aligned}$$

其中 $C(\cdot)$ 表示 token multiset，交集按 multiset 计数。候选片段对多个 peer 的 overlap score 为平均值：

$$\text{Overlap}(s) = \frac{1}{|P|} \sum_{p \in P} F_1(s, p).$$

然后加入长度正则和质量惩罚：

$$\text{Score}(s) = \text{Overlap}(s) + \text{BrevityPenalty}(s) + \text{QualityPenalty}(s).$$

长度正则用于惩罚与 peer 长度中位数差异过大的片段；质量惩罚用于惩罚 <think>、多次 answer boilerplate、空输出等异常内容。

每个模型有一个样本内权重 w_m ，初始为均匀分布：

$$w_m^{(0)} = \frac{1}{|M|}.$$

第 t 步选择最大化以下值的模型：

$$m_t^* = \arg \max_{m \in A_t} \left(\text{Score}(s_{m,t}) + \log w_m^{(t)} \right).$$

获胜片段 $s_{m_t^*,t}$ 被加入最终输出。随后用 softmax 后的得分作为 reward 更新权重：

$$\begin{aligned} r_m^{(t)} &= \frac{\exp(\text{Score}(s_{m,t}))}{\sum_{j \in A_t} \exp(\text{Score}(s_{j,t}))}, \\ \tilde{w}_m^{(t+1)} &= w_m^{(t)} \exp(\eta r_m^{(t)}), \\ w_m^{(t+1)} &= \frac{\tilde{w}_m^{(t+1)}}{\sum_j \tilde{w}_j^{(t+1)}}. \end{aligned}$$

最终输出为各步获胜片段拼接：

$$\hat{y}_{\text{SpecEM}} = s_{m_1^*,1} \oplus s_{m_2^*,2} \oplus \dots$$

如果所有片段选择失败，则退化为选择整体 overlap + 正则 + 权重最高的完整候选回答。

2.5 本次设置与原论文设置的系统性区别

表 1 汇总了原论文与本次复现之间的关键区别。

表 1: 原论文 SPECEM 与本次复现管线的关键区别。

维度	原论文设置	本次复现设置
实验范围	FuseEval 全量英文/中文场景, 以及 IFEval、MMLU、ARC-C、GSM8K、AlpacaEval 等多任务	只复现 FuseEval 开放生成核心部分, 使用 FuseEval-lite 400 条样本
数据来源	英文 Dolly-15k + Alpaca-GPT4; 中文 COIG-CQIA 中 Human-Value、Ruozb	英文来自 databricks/dolly 与 Alpaca 类来源; 中文来自 BAAI/COIG; 抽取 200 EN + 200 ZH
基础模型	Llama-3-8B、Mistral-7B-v0.3、Qwen2-7B、GLM-4-9B、Gemma-2-9B	MiMo-7B-SFT、Ministral-3-8B、Qwen3.5-9B、Gemma-3-12B-it
GLM	使用 GLM-4-9B-Instruct	未使用; 当前最新 GLM-5.1 为超大 MoE, 未找到适合单张 4090 的小参数最新版本
硬件	7B-9B 使用 A100; 大模型使用 H200	单张 RTX 4090 24GB
精度	bfloat16	4-bit NF4 量化, 顺序加载
模型驻留	多模型可并行参与推理	每次只加载一个模型; 先生成单模型输出, 再离线融合
算法形态	在线 drafting + verification + feedback	prefused 离线近似: 读取已生成完整输出并做片段选择
segment 参数	最大候选 segment 数 $L = 10$	segment token 长度为 48
反馈参数	$\alpha = 1$	$\eta = 0.35$
生成配置	do_sample=True, temperature 0.6, top-p 0.9	单模型脚本默认不采样; 验证脚本未传入 --do-sample; max new tokens 默认 256, MiMo 限制为 128
运行次数	三次独立运行平均	单次运行
中文分词	BLEU/ROUGE 前使用 Jieba	当前 sacreBLEU 使用默认 tokenization; ROUGE 用 rouge-score 直接计算

维度	原论文设置	本次复现设置
指标范围	BLEU、ROUGE、BARTScore、BERTScore、GPT4-Rank 等	sacreBLEU、ROUGE-1/2/L/Lsum；未使用 GPT judge/API 指标
质量控制	论文未以工程 quality gate 形式展开	明确检查行数、语言覆盖、空输出、thinking 泄漏、token cap
结论适用性	原论文完整 SPECEM 结论	本次单卡 prefused 近似结论，不能直接反驳或等同原论文

3 测评过程的详细数据管线是什么？

3.1 总体数据流

本次测评可以分为六个阶段：

1. 构建 FuseEval-lite 数据；
2. 为每个模型构建验证子集；
3. 单模型验证生成；
4. 质量门控；
5. 单模型完整 400 样本生成与评测；
6. SpecEM-4-prefused 融合与最终评测。

整体数据流如下：

原始数据源 → FuseEval-lite 400 → 验证子集 → 单模型验证输出 → 质量门控 → 单模型完整输出 → 单模型指标

3.2 数据样本格式

处理后的 FuseEval-lite 样本主要包含：

- `dataset`：样本来源或子集标签；
- `instruction`：输入给模型的任务指令；
- `output`：参考答案；
- 其他可能的辅助字段。

单模型输出 JSONL 在原样保留样本字段的基础上增加：

- `model_id`: 模型标识;
- `pred`: 模型生成答案;
- `generation_config`: 最大生成 token、是否采样、temperature、top-p 等;
- `elapsed_s`: 生成耗时;
- `new_tokens`: 新生成 token 数;
- `tokens_per_s`: 生成速度;
- `peak_vram_gb`: 峰值显存。

SpecEM 输出 JSONL 使用 `spec_pred` 存储融合结果，并额外记录：

- `source_models`: 参与融合四个模型;
- `dominant_model`: 该样本中片段贡献最多的模型;
- `candidate_lengths`: 各单模型候选输出长度;
- `trace`: 每一步片段选择、得分和权重更新记录。

3.3 验证子集与质量门控

每个模型先运行验证子集。验证子集默认每种语言 10 条，共 20 条。流程为：

1. 从 `data/processed/fuseeval_lite.json` 构建 `data/validation/<model>.json`;
2. 调用 `run_single_model.py` 生成验证输出;
3. 调用 `quality_gate_jsonl.py` 检查验证输出;
4. 验证通过后，才对完整 400 条样本运行该模型。

质量门控计算：

$$\begin{aligned} \text{empty_rate} &= \frac{\#\{\text{空输出}\}}{N}, \\ \text{thinking_rate} &= \frac{\#\{\text{包含 reasoning/thinking 标记的输出}\}}{N}, \\ \text{token_cap_rate} &= \frac{\#\{\text{生成 token 数达到上限的输出}\}}{N}. \end{aligned}$$

同时检查总行数是否等于预期，以及是否同时覆盖英文和中文样本。

3.4 单模型完整生成

验证通过后，每个模型运行完整 400 条样本。模型只在即将运行时下载或加载，运行完成后删除缓存以节省磁盘。完整输出位置为：

- results/raw/fuseeval_lite_gemma3_12b_it.jsonl
- results/raw/fuseeval_lite_mimo_7b_sft.jsonl
- results/raw/fuseeval_lite_qwen3_5_9b.jsonl
- results/raw/fuseeval_lite_ministral3_8b_instruct_2512.jsonl

每个单模型输出完成后，计算指标：

$$\{\hat{y}_i\}_{i=1}^N \quad \text{vs.} \quad \{y_i\}_{i=1}^N,$$

其中 $\hat{y}_i$ 是模型预测， y_i 是参考答案， $N = 400$ 。

3.5 SpecEM-4-prefused 融合

四个单模型输出全部完成并通过质量门控后，运行 `run_specem_prefused.py`。它首先按 `instruction` 对齐四个模型的输出：

$$\text{instruction}_i \mapsto \left(\hat{y}_i^{(1)}, \hat{y}_i^{(2)}, \hat{y}_i^{(3)}, \hat{y}_i^{(4)} \right).$$

如果某个模型缺少某条 `instruction`，会直接报错，因为融合要求样本严格对齐。

随后对每条样本执行前文所述片段融合，输出：

$$\hat{y}_i^{(\text{spec})}.$$

最终计算：

$$\{\hat{y}_i^{(\text{spec})}\}_{i=1}^{400} \quad \text{vs.} \quad \{y_i\}_{i=1}^{400}.$$

3.6 最终结果文件

最终 raw 输出均为 400 行 JSONL：

- Gemma3-12B-it: 400 行；
- MiMo-7B-SFT: 400 行；
- Qwen3.5-9B: 400 行；
- Ministral3-8B-Instruct-2512: 400 行；
- SpecEM-4-prefused: 400 行。

汇总指标文件为：

`specem_repro/results/metrics/fuseeval_lite_active_summary.json`.

4 几个指标分别代表什么？怎么算？数学公式是什么？

4.1 本次实际采用的指标

本次最终报告采用以下参考型词面指标：

- sacreBLEU；
- ROUGE-1；
- ROUGE-2；
- ROUGE-L；
- ROUGE-Lsum。

其中 sacreBLEU 是 BLEU 的标准化实现，主要衡量生成文本与参考文本的局部 n-gram 精确匹配；ROUGE 系列主要衡量生成文本对参考文本内容的覆盖和重叠。

4.2 BLEU / sacreBLEU

BLEU 的核心思想是：如果生成文本中大量 n-gram 出现在参考答案中，那么生成质量更高。它强调 precision，即“模型说出的内容有多少能在参考中找到”。这使得 BLEU 对短而贴近参考措辞的答案较友好，但会惩罚过度展开、换说法或风格变化。

设模型输出 corpus 为 C ，参考 corpus 为 R 。对 n-gram 阶数 n ，先计算 clipped precision：

$$p_n = \frac{\sum_{\text{ngram} \in C_n} \min(\text{count}_C(\text{ngram}), \text{count}_R(\text{ngram}))}{\sum_{\text{ngram} \in C_n} \text{count}_C(\text{ngram})}.$$

其中 C_n 表示系统输出中的所有 n-gram。

BLEU 同时加入 brevity penalty，防止模型输出极短答案来获得高 precision。设系统输出总长度为 c ，参考答案总长度为 r ：

$$BP = \begin{cases} 1, & c > r, \\ \exp(1 - r/c), & c \leq r. \end{cases}$$

标准 4-gram BLEU 为：

$$\text{BLEU} = BP \cdot \exp\left(\sum_{n=1}^4 w_n \log p_n\right),$$

通常 $w_n = \frac{1}{4}$ 。sacreBLEU 是 BLEU 的标准化实现，固定了 tokenization、大小写等实现细节，便于不同实验之间复现和比较。本次代码中调用：

```
sacrebleu.corpus_bleu(predictions, [references]).
```

本次复现中，Qwen3.5 的 sacreBLEU 最高，主要因为它的输出更短、更贴近参考答案局部措辞。指标审计显示：

- Qwen3.5: BLEU 10.0405, 长度比 0.9116, 4-gram precision 5.03;
- SpecEM-4-prefused: BLEU 5.8598, 长度比 1.7429, 4-gram precision 2.31。

SpecEM 输出更长, 虽然没有 brevity penalty, 但局部 n-gram precision 被稀释, 因此 BLEU 偏低。

4.3 ROUGE-N: ROUGE-1 与 ROUGE-2

ROUGE-N 衡量参考答案和生成答案之间的 n-gram 重叠。本次报告中的 ROUGE-1 使用 unigram, ROUGE-2 使用 bigram。

对于某条样本, 设参考文本为 R , 生成文本为 G 。令 $\mathcal{N}_n(R)$ 和 $\mathcal{N}_n(G)$ 分别为参考和生成中的 n-gram multiset。重叠数量为:

$$\text{overlap}_n = \sum_{g \in \mathcal{N}_n(R)} \min(\text{count}_R(g), \text{count}_G(g)).$$

ROUGE-N recall:

$$R_n = \frac{\text{overlap}_n}{\sum_{g \in \mathcal{N}_n(R)} \text{count}_R(g)}.$$

ROUGE-N precision:

$$P_n = \frac{\text{overlap}_n}{\sum_{g \in \mathcal{N}_n(G)} \text{count}_G(g)}.$$

ROUGE-N F1:

$$F_n = \frac{2P_n R_n}{P_n + R_n}.$$

本次实现使用 `rouge-score` 包, 报告的是每条样本 ROUGE F-measure 的平均值, 并乘以 100:

$$\text{ROUGE-N}_{\text{reported}} = 100 \cdot \frac{1}{N} \sum_{i=1}^N F_{n,i}.$$

ROUGE-1 反映词级内容覆盖; ROUGE-2 更严格, 要求相邻二元词组也匹配, 因此更能反映局部连贯短语是否接近参考答案。

4.4 ROUGE-L

ROUGE-L 基于最长公共子序列 (Longest Common Subsequence, LCS)。它不要求匹配词连续, 但要求顺序一致。因此, 它比 ROUGE-N 更关注整体顺序结构。

设参考文本 token 序列为 R , 生成文本 token 序列为 G , 最长公共子序列长度为:

$$\text{LCS}(R, G).$$

则:

$$R_{\text{LCS}} = \frac{\text{LCS}(R, G)}{|R|},$$

$$P_{LCS} = \frac{LCS(R, G)}{|G|}.$$

ROUGE-L F1 可写为：

$$F_{LCS} = \frac{2P_{LCS}R_{LCS}}{P_{LCS} + R_{LCS}}.$$

本次报告的 ROUGE-L 同样是逐样本 F1 平均后乘以 100：

$$\text{ROUGE-L}_{\text{reported}} = 100 \cdot \frac{1}{N} \sum_{i=1}^N F_{LCS,i}.$$

在本次实验中，SpecEM-4-prefused 的 ROUGE-L 最高，为 26.6490。这说明它在参考答案内容覆盖和顺序重叠方面优于单模型，但这不等价于 BLEU 更高，因为 BLEU 更依赖连续 n-gram precision。

4.5 ROUGE-Lsum

ROUGE-Lsum 是用于摘要或多句文本的 ROUGE-L 变体。它通常按句子边界处理文本，再计算 summary-level LCS。直观理解是：ROUGE-L 关注单条序列层面的最长公共子序列，而 ROUGE-Lsum 更适合多句生成结果，允许跨句级结构进行 summary-style 匹配。

在本次实现中，代码调用：

```
RougeScorer(["rouge1", "rouge2", "rougeL", "rougeLsum"], use_stemmer=True)
```

因此 ROUGE-Lsum 的具体细节由 rouge-score 包实现。本次报告值仍然是逐样本 F-measure 平均后乘以 100：

$$\text{ROUGE-Lsum}_{\text{reported}} = 100 \cdot \frac{1}{N} \sum_{i=1}^N F_{Lsum,i}.$$

本次 SpecEM-4-prefused 的 ROUGE-Lsum 为 27.6675，是所有系统中最高。这与其输出更长、更偏内容覆盖的特征一致。

4.6 质量门控指标

除最终参考指标外，本次复现还使用质量门控指标判断输出是否可信。

空输出率。

$$\text{empty_rate} = \frac{\#\{i : \hat{y}_i \text{ 为空或近似为空}\}}{N}.$$

该指标用于发现模型没有正常回答或解码失败。

reasoning 泄漏率。

$$\text{thinking_rate} = \frac{\#\{i : \hat{y}_i \text{ 包含 thinking/reasoning 标记}\}}{N}.$$

例如 <think>、thinking process、analyze the request 等。该指标尤其重要，因为 reasoning-tuned 模型可能把隐藏推理过程输出到最终答案中，导致评测结果失真。

token-cap 率。

$$\text{token_cap_rate} = \frac{\#\{i : \text{new_tokens}_i \geq T_{\max}\}}{N}.$$

该指标衡量多少样本生成到了最大 token 上限。token-cap 过高通常说明 EOS、stop token、chat template 或输出长度控制存在问题，也会影响 BLEU/ROUGE 解释。

4.7 原论文中涉及但本次未最终采用的指标

为避免混淆，需要区分原论文使用的指标和本次实际采用的指标。

BERTScore。 BERTScore 用上下文 embedding 衡量语义相似度，而非词面完全匹配。设生成 token embedding 为 $\{\hat{x}_i\}$ ，参考 token embedding 为 $\{x_j\}$ 。常见形式中：

$$P_{\text{BERT}} = \frac{1}{|\hat{x}|} \sum_i \max_j \cos(\hat{x}_i, x_j),$$

$$R_{\text{BERT}} = \frac{1}{|x|} \sum_j \max_i \cos(x_j, \hat{x}_i),$$

$$F_{\text{BERT}} = \frac{2P_{\text{BERT}}R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}}.$$

原论文报告 BERTScore F1。本次最终结果没有计算 BERTScore。

BARTScore。 BARTScore 使用生成式模型估计候选文本和参考文本之间的条件生成概率。一个常见方向是：

$$\text{BARTScore}(x, y) = \frac{1}{|y|} \sum_{t=1}^{|y|} \log p(y_t | y_{<t}, x).$$

它可以理解为用预训练 seq2seq 模型评估一个文本在另一个文本条件下的生成可能性。本次最终结果没有计算 BARTScore。

GPT4-Rank。 GPT4-Rank 是基于 GPT-4 或类似强模型 judge 的排序指标。它通常把多个模型回答提供给 judge，让 judge 根据质量排序，然后统计平均排名或 win-rate。原论文中 GPT4-Rank 数值越低越好。本次复现没有调用 GPT API，因此未计算该指标。

5 总结

FuseEval 是开放生成评测任务，核心是比较模型自由文本回答与参考回答的质量。本次复现使用 FuseEval-lite 400 条双语样本，在单张 RTX 4090 上用四个更新开源模型构建单模型基线，并实现了离线的 SpecEM-4-prefused 近似融合。该管线与原论文完整 SPECEM 在模型矩阵、硬件、精度、生成策略、在线/离线融合、segment 超参数、运行次数和中文分词协议上均存在重要差异，因此结果应解释为单卡近似复现结论。

指标上，sacreBLEU 更关注局部 n-gram precision，ROUGE 更关注与参考答案的内容重叠和覆盖。本次结果中，Qwen3.5 的 sacreBLEU 最高，SpecEM-4-prefused 的 ROUGE 最高；这并非简单 bug，而是输出长度、局部措辞精度和共识覆盖目标之间的张力造成的。